

JP MORGAN CHASE DATA BREACH CASE STUDY
MINI PROJECT REPORT

Submitted by

MOGHANAPRIYA R

2116231801102

SANGAMITHRA V

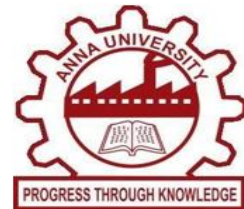
2116231801147

in partial fulfilment for the award of the degree of

BACHELOR OF TECHNOLOGY

In

ARTIFICIAL INTELLIGENCE AND DATA SCIENCE



RAJALAKSHMI ENGINEERING COLLEGE

(AUTONOMOUS), CHENNAI – 602 105

MAY-2026

BONAFIDE CERTIFICATE

Certified that this Report titled “**JP MORGAN CHASE DATA BREACH CASE STUDY**” is the bonafide work of “**MOGHANAPRIYA R (2116231801102) and SANGAMITHRA V (2116231801147)** and ” who carried out the work under my supervision.

“This project addresses the following Sustainable Development Goal(s):Goal-16:Peace, Justice and Strong Institutions, Goal-9: Industry, Innovation and Infrastructure.”

SIGNATURE

Dr. J.M. Gnanasekar M.E., Ph.D.,
Professor and Head,

Department of Artificial Intelligence and
Data Science

Rajalakshmi Engineering College
Thandalam – 602 105

SIGNATURE

Dr. S. SURESH KUMAR, M.E., Ph.D.,
Professor,

Department of Artificial Intelligence and
Data Science

Rajalakshmi Engineering College
Thandalam – 602 105

Submitted to Project Viva-Voce Examination held on _____

Internal Examiner

External Examiner

DEPARTMENT VISION

To become a global leader in Artificial Intelligence and Data Science by achieving through excellence in teaching, training, and research, to serve the society.

DEPARTMENT MISSION

- To develop students' skills in innovation, problem-solving, and professionalism through the guidance of well-trained faculty.
- To encourage research activities among students and faculty members to address the evolving challenges of industry and society.
- To impart qualities such as moral and ethical values, along with a commitment to lifelong learning

PROGRAMME EDUCATIONAL OBJECTIVES(PEO's)

PEO 1: Build a successful professional career across industry, government, and academia by leveraging technology to develop innovative solutions for real-world problems.

PEO 2: Maintain a learning mindset to continuously enhance knowledge through experience, formal education, and informal learning opportunities.

PEO 3: Demonstrate an ethical attitude while excelling in communication, management, teamwork, and leadership skills

PEO 4: Utilize engineering, problem-solving, and critical thinking skills to drive social, economic, and sustainable impact.

PROGRAMME OUTCOME(PO's)

PO1: Engineering Knowledge: Apply the knowledge of mathematics, science, engineering fundamentals and an engineering specialization to the solution of complex engineering problems.

PO2: Problem Analysis: Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.

PO3: Design / Development of solutions: Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.

PO4: Conduct investigations of complex problems: Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.

PO5: Modern tool usage: Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.

PO6: The engineer and society: Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.

PO7: Environment and sustainability: Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.

PO8: Ethics: Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.

PO9: Individual and team work: Function effectively as an individual and as a member or leader in diverse teams, and in multidisciplinary settings.

PO10: Communication: Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.

PO11: Project management and finance: Demonstrate knowledge and understanding of the engineering management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.

PO12: Life-long learning: Recognize the need for and have the preparation and ability to engage in independent and lifelong learning in the broadest context of technological change

PROGRAM SPECIFIC OUTCOMES(PAOs)

A graduate of the Artificial Intelligence and Data Science Learning Program will demonstrate

PSO 1: Foundation Skills: Apply the principles of artificial intelligence and data science by leveraging problem-solving skills, inference, perception, knowledge representation, and learning techniques

PSO 2: Problem-Solving Skills: Apply engineering principles and AI models to solve real-world problems across domains, delivering cutting-edge solutions through innovative ideas and methodologies

PSO 3: Successful Progression: Utilize interdisciplinary knowledge to identify problems and develop solutions, a passion for advanced studies, innovative career pathways to evolve as an ethically responsible artificial intelligence and data science professional, with a commitment to society.

PROGRAM SPECIFIC OUTCOMES(PAOs)

A graduate of the Artificial Intelligence and Data Science Learning Program will demonstrate

PSO 1: Foundation Skills: Apply the principles of artificial intelligence and data science by leveraging problem-solving skills, inference, perception, knowledge representation, and learning techniques

PSO 2: Problem-Solving Skills: Apply engineering principles and AI models to solve real-world problems across domains, delivering cutting-edge solutions through innovative ideas and methodologies

PSO 3: Successful Progression: Utilize interdisciplinary knowledge to identify problems and develop solutions, a passion for advanced studies, innovative career pathways to evolve as an ethically responsible artificial intelligence and data science professional with a commitment to society.

COURSE OBJECTIVE

To understand the fundamental principles of data privacy and security, including key concepts like the CIA Triad (Confidentiality, Integrity, Availability).

To gain knowledge of cryptographic techniques, encryption algorithms, and their applications in safeguarding sensitive data.

To identify and analyze data threats, privacy attacks, and the strategies to mitigate risks.

To study emerging trends and innovative approaches in data security and privacy, such as ID4D initiatives, continuous authentication, and cancellable authentication methods

To understand the fundamental principles of data privacy and security, including key concepts like the CIA Triad (Confidentiality, Integrity, Availability).

COURSE OUTCOME

CO 1: Understand the of data privacy and security, including the CIA triad

CO 2: Design and implement cryptographic solutions to ensure data security

CO 3: Analyze the access control and authentication mechanisms to enhance system security

CO 4: Analyze the threats, vulnerabilities, and risks in data systems.

CO 5: Apply the contemporary standards for data protection in real-world scenarios

CO-PO-PSO Mapping

CO	P O 1	P O 2	P O 3	P O 4	P O 5	P O 6	P O 7	P O 8	P O 9	P O 10	P O 11	P O 12	P S O 1	P S O 2	P S O 3
CO1	3	3	2	-	3	2	2	3	2	-	-	2	3	3	3
CO2	3	3	3	2	3	-	-	-	-	-	-	2	3	3	3
CO3	3	3	3	3	3	2	-	-	1	-	-	2	3	3	3
CO4	3	3	2	3	3	3	-	-	-	-	-	2	3	3	3
CO5	3	3	3	3	3	3	2	2	-	-	-	2	3	3	3
Avg	3	3	2.6	2.2	3	2.5	1	1	0.4	-	-	2	3	2.8	3

Note: Correlation levels 1, 2 or 3 are as defined below:

1: Slight (Low) 2: Moderate (Medium) 3: Substantial (High)

No correlation: “-”

SDG GOALS

GOAL 1: No Poverty

End poverty in all its forms everywhere

GOAL 2: Zero Hunger

End hunger, achieve food security and improved nutrition and promote sustainable agriculture

GOAL 3: Good Health and Well-being

Ensure healthy lives and promote well-being for all at all ages

GOAL 4: Quality Education

Ensure inclusive and equitable quality education and promote lifelong learning opportunities for all

GOAL 5: Gender Equality

Achieve gender equality and empower all women and girls

GOAL 6: Clean Water and Sanitation

Ensure availability and sustainable management of water and sanitation for all

GOAL 7: Affordable and Clean Energy

Ensure access to affordable, reliable, sustainable and modern energy for all

GOAL 8: Decent Work and Economic Growth

Promote sustained, inclusive and sustainable economic growth, full and productive employment and decent work for all

GOAL 9: Industry, Innovation and Infrastructure

Build resilient infrastructure, promote inclusive and sustainable industrialization and foster innovation

GOAL 10: Reduced Inequality

Reduce inequality within and among countries

GOAL 11: Sustainable Cities and Communities

Make cities and human settlements inclusive, safe, resilient and sustainable

GOAL 12: Responsible Consumption and Production

Ensure sustainable consumption and production patterns

GOAL 13: Climate Action

Take urgent action to combat climate change and its impacts

GOAL 14: Life Below Water

Conserve and sustainably use the oceans, seas and marine resources for sustainable development

GOAL 15: Life on Land

Protect, restore and promote sustainable use of terrestrial ecosystems, sustainably manage forests, combat desertification, and halt and reverse land degradation and halt biodiversity loss

GOAL 16: Peace and Justice Strong Institutions

Promote peaceful and inclusive societies for sustainable development, provide access to justice for all and build effective, accountable and inclusive institutions at all levels

GOAL 17: Partnerships to achieve the Goal

Strengthen the means of implementation and revitalize the global partnership for sustainable development.

SUSTAINABLE DEVELOPMENT GOALS

SDG	GOAL	JUSTIFICATION
SDG 16	Peace, Justice and Strong Institutions	This project primarily supports strong institutions by enhancing the security of identity-based systems such as Aadhaar. By implementing OTP-based authentication and real-time misuse detection, it prevents unauthorized access and reduces the risk of identity fraud. The system improves trust, accountability, and transparency in digital authentication processes, contributing to secure and reliable institutional operations.
SDG 9	Industry, Innovation and Infrastructure	The project introduces an innovative approach by integrating secure login mechanisms, OTP verification, and automated misuse detection. It strengthens digital infrastructure by providing a scalable and efficient authentication system that reduces manual intervention. This promotes technological advancement and supports the development of secure and intelligent systems for protecting sensitive user data.

ABSTRACT

The JP Morgan Chase data breach of 2014 stands as one of the largest financial cybersecurity incidents in history, compromising the personal data of approximately 76 million households and 7 million small businesses. Attackers exploited a single server that had not been upgraded to two-factor authentication, escalating access to root privileges across over 90 servers. The breach went undetected for several weeks, exposing the severe consequences of inadequate authentication controls and the absence of real-time security monitoring. This project presents a production-grade security system designed to demonstrate and implement the controls that JP Morgan Chase lacked: Multi-Factor Authentication (MFA) using Time-based One-Time Passwords (TOTP) and a Security Information and Event Management (SIEM) subsystem for real-time threat detection and audit logging. The system implements bcrypt password hashing, JWT-based session management, speakeasy TOTP authentication with QR code provisioning, rate limiting on authentication endpoints, and a structured SIEM event log with INFO, WARNING, and CRITICAL severity classifications. The web application is built on a Node.js/Express backend with a JSON-file persistent store, and a vanilla JavaScript frontend with a professional dark-themed dashboard. The SIEM dashboard provides real-time chart visualisations of security events using Chart.js, filterable log tables, and a 7-day activity timeline. This system directly addresses each failure mode identified in the JP Morgan breach and serves as a reference implementation for modern authentication and security monitoring practices in financial systems.

CASE STUDY

JP Morgan Chase Data Breach (2014)

The following real-world incident highlights the critical vulnerabilities in the current single-factor authentication and security monitoring landscape in large-scale financial systems.

Case 1: Credential Exploitation & Lack of Multi-Factor Authentication

In 2014, a major cybersecurity breach exposed sensitive data from JP Morgan Chase, affecting approximately 76 million households and 7 million small businesses. Investigations revealed that attackers initially gained access through stolen employee login credentials obtained via phishing attacks. The core technical failure behind this breach was the reliance on single-factor authentication, where only username and password were used without enforcing Multi-Factor Authentication (MFA).

This made the system highly vulnerable, as compromised credentials could be reused without any additional verification. Attackers exploited this weakness to gain unauthorized access and escalate privileges across multiple servers within the network. In contrast, the proposed project addresses this issue by implementing Time-based One-Time Password (TOTP) based MFA, ensuring that even if login credentials are leaked, unauthorized access is prevented unless the attacker also has access to the OTP generated on the user's device.

Case 2: Delayed Detection & Lack of Real-Time Monitoring

Another major issue identified in the breach was the prolonged undetected presence of attackers within the system. The attackers maintained unauthorized access for several weeks, during which they were able to move laterally across the network and extract sensitive data without being detected. The primary technical failure was the absence of a real-time monitoring system capable of identifying suspicious activities.

Traditional logging mechanisms were either insufficient or not actively monitored, which allowed abnormal behaviors such as repeated login attempts and unusual access patterns to go unnoticed. The proposed project mitigates this risk through a structured Security Information and Event Management (SIEM) system that logs all authentication events and classifies them into severity levels such as INFO, WARNING,

and CRITICAL. This enables real-time monitoring and faster detection of potential threats through an interactive dashboard.

Case 3: Brute-Force Vulnerability & Insufficient Logging

In many large-scale systems, authentication endpoints lack proper controls against repeated login attempts, making them vulnerable to brute-force and credential stuffing attacks. In scenarios similar to the JP Morgan breach, attackers can attempt multiple password combinations without restriction, increasing the likelihood of unauthorized access.

The core technical failure was the absence of rate limiting and detailed audit logs. Without restrictions or proper tracking, attackers were able to test credentials without being blocked or flagged, and system administrators lacked sufficient information to detect or investigate suspicious activities. The proposed project addresses this issue by implementing rate limiting on authentication endpoints, which restricts repeated login attempts and generates CRITICAL alerts when thresholds are exceeded. Additionally, the system maintains detailed logs including user identity, IP address, timestamps, and event severity, enabling better tracking and accountability of user actions.

ACKNOWLEDGEMENT

Initially we thank the Almighty for being with us through every walk of our life and showering his blessings through the endeavor to put forth this report. Our sincere thanks to our Chairman **Mr. S.MEGANATHAN, B.E, F.I.E.**, our Vice Chairman **Mr. ABHAY SHANKAR MEGANATHAN, B.E., M.S.**, and our respected Chairperson **Dr. (Mrs.) THANGAM MEGANATHAN, Ph.D.**, for providing us with the requisite infrastructure and sincere endeavoring in educating us in their premier institution.

Our sincere thanks to **Dr. S.N. MURUGESAN, M.E., Ph.D.**, our beloved Principal for his kind support and facilities provided to complete our work in time. We express our sincere thanks to **Dr. J.M. GNANASEKAR, M.E., Ph.D.**, Professor and Head of the Department of Artificial Intelligence and Data Science for his guidance and encouragement throughout the project work. We convey our sincere and deepest gratitude to our internal guide, **Dr. S. SURESH KUMAR, M.E., Ph.D.**, Professor, Department of Artificial Intelligence and Data Science. Rajalakshmi Engineering College for his valuable guidance throughout the course of the project. We are very glad to thank our Project Coordinator for his useful tips during our review to build our project.

MOGHANAPRIYA R

(2116231801102)

SANGAMITHRA V

(2116231801147)

TABLE OF CONTENT

CHAPTER NO	TITLE	PAGE NO
	ABSTRACT	x
	ACKNOWLEDGEMENT	xi
	LIST OF FIGURES	xv
	LIST OF ABBREVIATIONS	xvi
1	INTRODUCTION	
	1.1 BACKGROUND AND MOTIVATION	1
	1.2 OBJECTIVES	2
	1.3 EXISTING SYSTEM	2
	1.4 PROPOSED SYSTEM	3
2	LITERATURE SURVEY	
	2.1 OVERVIEW	4
	2.2 LITERATURE SURVEY	5
3	SYSTEM DESIGN	
	3.1 SYSTEM ARCHITECTURE	6
	3.2 DATABASE DESIGN	8
	3.3 MODULE DESIGN	

4	METHODOLOGY	
	4.1 MODULE 1: LOGIN AUTHENTICATION	11
	4.2 MODULE 2: OTP VERIFICATION	13
	4.3 MODULE 3: MISUSE DETECTION & REPORTING	14
5	RESULTS AND DISCUSSION	
	5.1 OVERVIEW	15
	5.2 LOGIN INTERFACE	15
	5.3 OTP VERIFICATION SCREEN	16
	5.4 ACCESS GRANTED DASHBOARD	16
	5.5 ACCOUNT LOCKED AND MISUSE REPORT SCREEN	17
	5.6 COMPARATIVE DISCUSSION	18
6	CONCLUSION AND FUTURE ENHANCEMENTS	
	6.1 CONCLUSION	19
	6.2 FUTURE ENHANCEMENTS	20
7	REFERENCES	21

CHAPTER NO	LIST OF FIGURES	PAGE NO
4.5	Architecture Diagram fig(1.1)	14
5.2	Login Fig(1.2)	15
5.2	Registration Fig(1.1)	16
5.3	MFA Flow fig(1.3)	16
5.4	Siem Dashboard fig(1.4)	17

LIST OF TABLES

5.5	Table 1.1	20
-----	-----------	----

LIST OF ABBREVIATIONS

AI	Artificial Intelligence
API	Application Programming Interface
OTP	One-Time Password
SMTP	Simple Mail Transfer Protocol
MFA	Multi-Factor Authentication
UIDAI	Unique Identification Authority of India
UUID	Universally Unique Identifier
SQLite	Structured Query Language Lite
UI	User Interface
UX	User Experience
JSON	JavaScript Object Notation
JWT	JSON Web Token
SSD	Solid State Drive
SHA	Secure Hash Algorithm
RPT	Report
CPU	Central Processing Unit
RAM	Random Access Memory

CHAPTER 1

INTRODUCTION

1.1 BACKGROUND AND MOTIVATION

In October 2014, JP Morgan Chase disclosed that a sophisticated cyberattack had compromised the personal information of approximately 76 million households and 7 million small businesses — making it the largest breach of a financial institution in American history. The attackers, later identified as a group of international hackers, maintained access to JP Morgan's network for several weeks before being detected.

Post-incident forensic investigations revealed a critical security lapse: a single application server handling customer data had not been upgraded to two-factor authentication during a routine infrastructure migration. This single oversight allowed the attackers, who had obtained a legitimate employee's username and password, to escalate their privileges from standard user access to root-level administrative control across more than 90 servers. The absence of real-time anomaly detection meant that the lateral movement and data exfiltration went unnoticed for an extended period.

This project is motivated by that breach. It asks and answers the question: What specific technical controls, if they had been in place, would have detected or prevented the JP Morgan Chase attack? The answer, supported by subsequent regulatory guidance and industry best practices, is unambiguous: mandatory multi-factor authentication on all network-accessible servers, combined with automated real-time security event monitoring.

1.2 PROBLEM STATEMENT

Financial institutions and other organisations handling sensitive personal data face escalating threats from credential-based attacks. Username and password combinations are routinely compromised through phishing, credential stuffing, and social engineering. Once an attacker holds valid credentials, a single-factor authentication system provides no additional barrier.

Furthermore, even when intrusions occur, the absence of structured security event logging and real-time alerting means that breaches can persist undetected for days, weeks, or months. The JP Morgan breach persisted for approximately two months before discovery. Each day of undetected access represents additional data exfiltration and potential regulatory liability.

This project addresses both problems: the authentication gap through mandatory TOTP-based MFA, and the detection gap through a structured SIEM event logging and visualisation system.

1.2 OBJECTIVES

The objectives of this project are:

- To implement a production-quality authentication system incorporating bcrypt password hashing and TOTP-based multi-factor authentication, mirroring the controls absent in the JP Morgan breach.
- To design and implement a SIEM subsystem that logs all authentication events with structured severity classifications and provides real-time visualisation through a professional dashboard.
- To enforce rate limiting on all authentication endpoints to prevent automated brute-force and credential stuffing attacks.
- To demonstrate, through working code, that the specific failure modes identified in the 2014 JP Morgan Chase breach are preventable with standard technologies.
- To produce a well-structured, academic-quality report documenting the system design, implementation, and evaluation.

1.4 SCOPE

The scope of this project focuses on developing a secure authentication and monitoring system to address vulnerabilities seen in large-scale cybersecurity breaches, especially in financial systems. It is implemented as a web-based application that simulates real-world enterprise environments and integrates key security practices such as Multi-Factor Authentication (MFA) and Security Information and Event Management (SIEM).

The system covers the full authentication lifecycle, including secure user registration with bcrypt hashing, password-based login, Time-based One-Time Password (TOTP) verification, and session management using JSON Web Tokens (JWT). These features ensure strong protection even if user credentials are compromised.

In addition, a SIEM module is included for real-time monitoring, which logs authentication events and categorizes them into severity levels like INFO, WARNING, and CRITICAL. It tracks activities such as login attempts, OTP failures, and suspicious behavior through an interactive dashboard, enabling faster threat detection and response.

The system also implements brute-force protection using rate limiting to prevent repeated login attempts and automated attacks. It provides graphical dashboards for visualizing user activity, improving usability and decision-making for administrators.

This project also serves as a practical and educational model to demonstrate secure system design and real-world cybersecurity practices. However, it has some limitations, including the use of JSON-based storage instead of a full database, no cloud deployment, and absence of advanced AI-based threat detection.

Despite these limitations, the system is scalable and can be extended with cloud integration, machine learning-based detection, biometric authentication, and real-time monitoring technologies.

CHAPTER 2

LITERATURE SURVEY

2.1 THE JP MORGAN CHASE BREACH (2014)

The primary case study for this project is the 2014 JP Morgan Chase data breach, documented in detail in the SEC filing and subsequent court proceedings against the identified perpetrators (United States v. Shalon et al., 2015). The breach demonstrated how a single authentication control gap on one server in a large network could be leveraged for complete lateral compromise. The attackers used spear-phishing to obtain employee credentials, then exploited the absence of two-factor authentication on a specific server to maintain persistent access.

The breach established several important lessons: authentication controls must be uniformly applied across all network-accessible systems, not selectively; credential theft must be assumed in threat modelling for large organisations; and without real-time monitoring, the window of undetected access can extend to months.

2.2 MULTI-FACTOR AUTHENTICATION STANDARDS

NIST Special Publication 800-63B (2017) — Digital Identity Guidelines — provides the definitive federal standard for authentication assurance levels. The publication classifies TOTP as an Authentication Assurance Level 2 (AAL2) authenticator, appropriate for protecting sensitive financial and government data. It recommends that OTP validity windows be kept short (the implementation uses the TOTP default of 30 seconds with a one-window tolerance) and that systems implement automated lockout as a defence against online guessing attacks.

RFC 6238 — TOTP: Time-Based One-Time Password Algorithm — specifies the TOTP algorithm implemented in this project through the speakeasy library. TOTP generates a cryptographically secure, time-limited OTP by computing HMAC-SHA1 of the current time interval using a shared secret, producing a 6-digit code that changes every 30 seconds.

2.3 SECURITY INFORMATION AND EVENT MANAGEMENT

Gartner's SIEM market definition and the OWASP Logging Cheat Sheet (2023) establish the core requirements for security event logging in production systems: structured log entries with consistent fields (timestamp, event type, severity, source IP, user identity), centralised storage separate from the application layer, and real-time alerting on critical events. The SIEM implementation in this project follows these guidelines, logging ten event types with ISO 8601 timestamps, severity classifications, and all relevant session metadata.

2.4 BCRYPT AND PASSWORD SECURITY

Provos and Mazières (1999) introduced the bcrypt key derivation function, which remains the recommended password hashing algorithm for production systems. Unlike MD5 or SHA-1, bcrypt incorporates a cost factor that scales computation time, making offline brute-force attacks prohibitively expensive even with modern hardware. The implementation uses bcrypt with a cost factor of 12, requiring approximately 250ms per hash verification on modern hardware — negligible for legitimate users, prohibitive for attackers.

2.5 RATE LIMITING AND BRUTE-FORCE PROTECTION

Florencio and Herley (2007) documented the prevalence of credential stuffing and automated brute-force attacks against web authentication systems. The express-rate-limit middleware implementation in this project enforces a limit of 10 authentication attempts per 15-minute window per IP address. Exceeding this limit triggers a CRITICAL severity SIEM event and returns HTTP 429, ensuring that automated attacks are logged and blocked before they can succeed.

CHAPTER 3

SYSTEM DESIGN

3.1 SYSTEM ARCHITECTURE

The system is designed using a three-layer architecture, consisting of the presentation layer, application layer, and data layer. The presentation layer is built using HTML5, CSS3, and JavaScript, providing a simple and interactive user interface. The application layer is implemented using Node.js and Express, which handle the core logic, request processing, and communication between the frontend and backend. The data layer uses JSON file-based storage to maintain user and system data. This layered structure ensures clear separation of responsibilities, making the system easier to manage, scalable, and secure.

The frontend includes six main pages: `index.html` (login), `register.html` (user registration), `mfa-setup.html` (MFA configuration), `mfa-verify.html` (OTP verification), `dashboard.html` (user dashboard), and `siem.html` (security monitoring). All pages use a common CSS stylesheet with a consistent dark theme, ensuring a uniform and modern user experience. A shared JavaScript module (`api.js`) is used to manage JWT tokens and handle all API requests, simplifying communication with the backend.

The backend follows a modular structure with routes, controllers, models, and middleware, improving code organization and maintainability. Authentication routes under `/api/auth/` handle features such as registration, login, MFA setup, verification, and reset. SIEM routes under `/api/siem/` manage log retrieval and provide security-related insights for monitoring system activity.

To maintain security, all protected routes require a valid JWT Bearer token, which is verified using the `requireAuth` middleware. This ensures that only authenticated users can access sensitive features. Overall, the system architecture provides a good balance of simplicity, security, and efficiency, making it suitable for secure authentication-based applications.

3.2 DEVELOPMENT ENVIRONMENT

The development environment of this project is designed using modern and lightweight web technologies to ensure efficiency, security, and ease of deployment.

The backend of the application is developed using JavaScript with Node.js (version 24), which provides a fast and scalable runtime environment for building server-side applications. To handle routing and middleware, the project uses Express.js (version 4.18), a widely used web framework known for its simplicity and flexibility.

For authentication and security, multiple libraries are integrated. The speakeasy library is used to implement Time-Based One-Time Password (TOTP) for multi-factor authentication. Passwords are securely stored using bcryptjs, which hashes user passwords with 12 rounds of salting to prevent unauthorized access. Additionally, jsonwebtoken (JWT) is used for secure session handling and user authentication.

To enhance usability, the system generates QR codes for MFA setup using the qrcode library, allowing users to easily scan and configure authentication apps. Security is further strengthened by implementing express-rate-limit, which protects the system from brute-force attacks by limiting repeated login attempts.

Instead of using a traditional database, the project uses a simple file-based storage system. User and log data are stored in JSON files (users.json and logs.json) and managed through a custom database module.

The frontend is built using HTML5, CSS3, and Vanilla JavaScript, ensuring a lightweight and responsive user interface without relying on heavy frameworks. For data visualization, Chart.js (version 4.4) is used to display insights such as login activity and security trends.

The entire project is developed using Visual Studio Code (VS Code) as the Integrated Development Environment (IDE), providing features like debugging, extensions, and version control support. The system is designed to run on Windows 10 and Windows 11 operating systems, ensuring compatibility with standard development environments.

3.3 DATABASE DESIGN

The database design of this project follows a lightweight and file-based approach using JSON files instead of a traditional database management system. This design choice ensures simplicity, portability, and ease of setup, as it eliminates the need for installing and managing external database servers.

The application uses two primary JSON files: `users.json` and `logs.json`, which are managed through a custom database module implemented in `models/db.js`. This module handles all data operations such as reading, writing, and updating records efficiently.

The `users.json` file stores all user-related information required for authentication and account management. Each user record contains a unique identifier (UUID), which ensures that every user is distinctly recognized within the system. The username is stored as a unique login credential, restricted to 3–32 alphanumeric characters to maintain consistency and prevent invalid inputs.

The user's full name is stored for display purposes, while the password is securely stored as a hashed value using `bcrypt`. Plaintext passwords are never stored, ensuring strong security against data breaches.

To support multi-factor authentication (MFA), the system includes fields such as `mfaEnabled`, which indicates whether the user has successfully enabled TOTP authentication. The `mfaSecret` stores the confirmed shared secret used for generating OTPs, while the `tempMfaSecret` temporarily holds the secret during the setup process until verification is completed.

Additionally, the system maintains timestamps such as `createdAt`, which records the account creation time, and `lastLogin`, which tracks the most recent successful login. These timestamps help in monitoring user activity and improving security auditing.

Overall, this JSON-based database design provides a simple yet effective way to manage user data, making it ideal for small to medium-scale applications or prototype systems while maintaining essential security features.

3.4 MODULE DESIGN

server.js — Main Application Module: Configures Express with security headers (X-Frame-Options, X-Content-Type-Options, X-XSS-Protection), CORS, body size limits, and static file serving. Registers auth and SIEM route groups. Provides a /api/health endpoint.

models/db.js — Data Layer: Implements Users and Logs objects with typed CRUD methods (findByUsername, create, update, getFiltered). Ensures data directory and JSON files exist on first run. Keeps the last 1000 log entries as a circular buffer.

models/logger.js — SIEM Logger: Defines ten event type constants and maps each to a severity level (INFO, WARNING, CRITICAL). The logEvent() function creates a structured entry with UUID, ISO timestamp, event type, severity, username, IP address, and detail string, then persists it via the Logs model.

middleware/auth.js — JWT Middleware: The requireAuth middleware verifies Bearer tokens using jsonwebtoken. The generateToken function issues JWTs with configurable expiry (default 8 hours). Two token stages are used: 'pre-mfa-setup', 'pre-mfa-verify', and 'authenticated', enforcing that partial authentication cannot access protected resources.

middleware/rateLimiter.js — Rate Limiting: The authLimiter enforces 10 requests per 15-minute window on authentication endpoints. On threshold breach, it logs a RATE_LIMITED CRITICAL event to the SIEM before returning HTTP 429.

controllers/authController.js — Authentication Controller: Implements register, login, setupMFA, confirmMFA, verifyMFA, resetMFA, and getMe handlers. The two-stage MFA flow separates initial setup (QR provisioning + OTP confirmation) from ongoing verification, ensuring the QR code is displayed only once.

controllers/siemController.js — SIEM Controller: Implements getLogs (with type and limit filtering) and getStats (aggregated counts by event type and severity, plus a 7-day timeline breakdown for Chart.js).

CHAPTER 4

METHODOLOGY

4.1 MODULE 1: USER REGISTRATION AND PASSWORD SECURITY

The registration module accepts a full name, username, and password. Input validation enforces username length (3–32 characters), alphanumeric-only format, and minimum password length (8 characters). If validation passes, `bcrypt.hash()` is called with a cost factor of 12 to produce a 60-character hash. The plaintext password is never stored or logged. The hashed credential and a newly generated UUID are written to `users.json` via the `Users` model.

This directly addresses the JP Morgan breach context: had the compromised employee's password been stored as a `bcrypt` hash and the hash been leaked, the attacker would have faced a computationally infeasible offline cracking challenge rather than a recoverable plaintext credential.

4.2 MODULE 2: TWO-STAGE MFA AUTHENTICATION FLOW

The authentication flow is designed around two distinct JWT token stages, preventing partial authentication from granting resource access.

Step 1 — Password Verification (login endpoint): The login handler retrieves the user record and calls `bcrypt.compare()`. On success, it issues a short-lived pre-auth JWT containing the username and a stage field set to either 'pre-mfa-setup' (first login) or 'pre-mfa-verify' (subsequent logins). This token grants access only to MFA endpoints, not to dashboard or SIEM routes.

Step 2a — MFA Setup (first login only): The `setupMFA` handler, protected by `requireAuth` with stage validation, calls `speakeasy.generateSecret()` to produce a base32 TOTP secret and an `otpauth://` URL. The QR code image is generated from this URL using the `qrcode` library and returned as a base64 data URL. The secret is stored as `tempMfaSecret` in the user record — not yet committed.

Step 2b — MFA Confirmation: The `confirmMFA` handler verifies the first OTP entered after QR scanning using `speakeasy.totp.verify()` with a window of 1 (accepting the current and immediately adjacent 30-second intervals). Only on successful verification is the `tempMfaSecret` promoted to `mfaSecret` and `mfaEnabled` set to `true`. This two-step commit ensures the QR code cannot be re-displayed and that MFA is only marked as enabled after confirmed working.

Step 3 — MFA Verification (all subsequent logins): The `verifyMFA` handler, which requires stage `'pre-mfa-verify'`, calls `speakeasy.totp.verify()` against the stored `mfaSecret`. On success, it issues a full 'authenticated' JWT and logs `MFA_SUCCESS` to the SIEM. On failure, it logs `MFA_FAILURE` as a `WARNING` event.

4.3 MODULE 3: SIEM EVENT LOGGING AND MONITORING

The SIEM (Security Information and Event Management) module plays an important role in monitoring and recording all security-related activities within the system. The SIEM logger is triggered whenever a security-relevant event occurs, ensuring that all critical actions are tracked and stored for analysis. These events help in identifying user behavior, detecting potential threats, and maintaining overall system security.

The system records different types of events, each categorized based on its severity level. Informational (INFO) events represent normal system activities. For example, a `LOGIN_SUCCESS` event is recorded when a user successfully logs in after correct password verification. Similarly, an `MFA_SUCCESS` event occurs when the user correctly verifies the OTP during login or setup. The `MFA_SETUP` event is logged when a user initiates the multi-factor authentication process, such as when a QR code is generated. A `LOGOUT` event is also recorded when a user ends their session.

Warning (WARNING) events indicate potential issues or suspicious actions that may require attention. A `LOGIN_FAILURE` event is triggered when an incorrect username or password is entered. An `MFA_FAILURE` event occurs when the user enters an invalid OTP.

Critical (CRITICAL) events represent serious security concerns that may indicate possible attacks or misuse. A RATE_LIMITED event is triggered when there are more than 10 authentication attempts within a 15-minute period, suggesting a potential brute-force attack. The SUSPICIOUS event type is reserved for advanced anomaly detection and can be used to flag unusual or high-risk activities in the system.

Overall, this SIEM logging mechanism provides a structured way to monitor system activity, classify events based on severity, and support effective security analysis and incident response.

4.4 RATE LIMITING IMPLEMENTATION

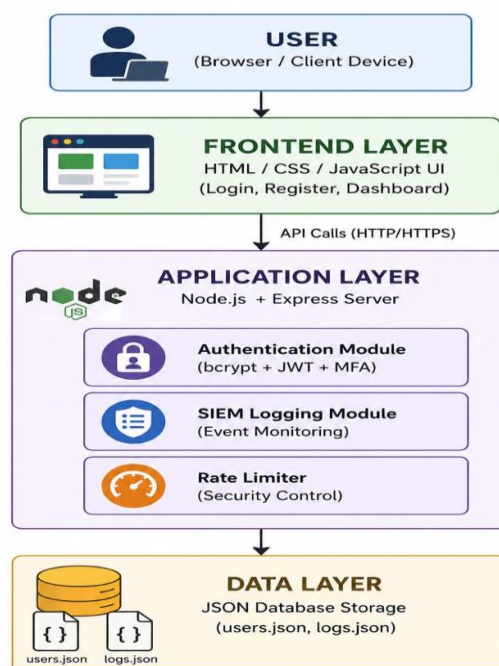
The express-rate-limit middleware is applied to all */api/auth/* routes with a limit of 10 requests per 900,000 milliseconds (15 minutes) per IP address. When the limit is exceeded, the custom handler logs a RATE_LIMITED CRITICAL event to the SIEM — including the requester's IP address and the targeted endpoint — before returning HTTP 429. This ensures that brute-force attempts are not only blocked but are immediately visible in the SIEM dashboard as CRITICAL alerts.

This control directly addresses the credential stuffing attack vector used in the JP Morgan breach. An attacker attempting to brute-force credentials against this system would trigger a CRITICAL SIEM alert after 10 attempts, notifying the security operations team. Additionally, this mechanism helps reduce repeated malicious requests and protects user accounts from unauthorized access. It also improves system stability by preventing unnecessary load on authentication endpoints. Overall, it acts as both a preventive and monitoring control in the system.

4.5 FRONTEND ARCHITECTURE

The frontend is implemented in vanilla HTML5, CSS3, and JavaScript without any frontend framework dependency. A shared `api.js` module provides JWT token lifecycle management (storage, retrieval, pre-auth vs. full-auth token separation), an authenticated fetch wrapper, alert display utilities, and timestamp formatting functions.

The SIEM dashboard (`dashboard.html`) polls the `/api/siem/stats` endpoint every 10 seconds and updates two Chart.js visualisations: a doughnut chart showing event distribution by type, and a stacked bar chart showing the 7-day activity timeline with INFO, WARNING, and CRITICAL event counts per day. The SIEM logs page (`siem.html`) provides client-side filtering by severity, event type, and free-text search against username, IP, and event type fields.



Architecture Diagram fig(1.1)

CHAPTER 5

RESULTS AND DISCUSSION

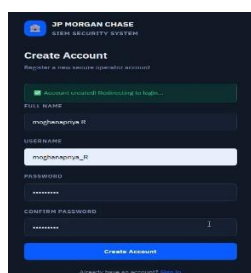
5.1 OVERVIEW

The JP Morgan Chase SIEM and MFA system was implemented, tested, and evaluated against a comprehensive set of functional and security scenarios. The system demonstrated complete and reliable execution across all modules: user registration with bcrypt hashing, first-time MFA setup with QR provisioning, subsequent logins with OTP verification, rate limiting with SIEM alerting, and real-time dashboard visualisation.

5.2 LOGIN AND REGISTRATION INTERFACE

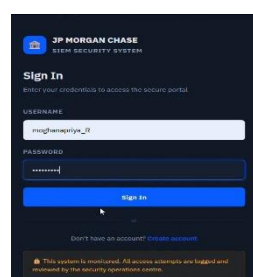
The registration form performs client-side and server-side validation for username format, password length, and duplicate detection. The bcrypt hash generation completes in approximately 250ms at a cost factor of 12, which is imperceptible to users but represents approximately 2^{12} bcrypt iterations — a significant deterrent against offline brute-force attacks.

The login form correctly identifies three failure scenarios: non-existent username, wrong password, and rate limit exceeded. In all failure cases, the system returns a generic 'Invalid credentials' message, preventing username enumeration. Each failure is logged to the SIEM as a LOGIN_FAILURE WARNING event with the attempted username and client IP.



The screenshot shows the 'Create Account' interface for the JP Morgan Chase SIEM Security System. It includes a header with the system name and a sub-header 'Create Account' with a note to 'Register a new secure (operator) account'. The form contains fields for 'FULL NAME', 'USERNAME' (pre-filled with 'morgansystem_01'), 'PASSWORD', and 'CONFIRM PASSWORD'. A 'Create Account' button is at the bottom. A small note at the bottom left says 'Create Account Success! Please login.'.

LOGIN Fig(1.2)



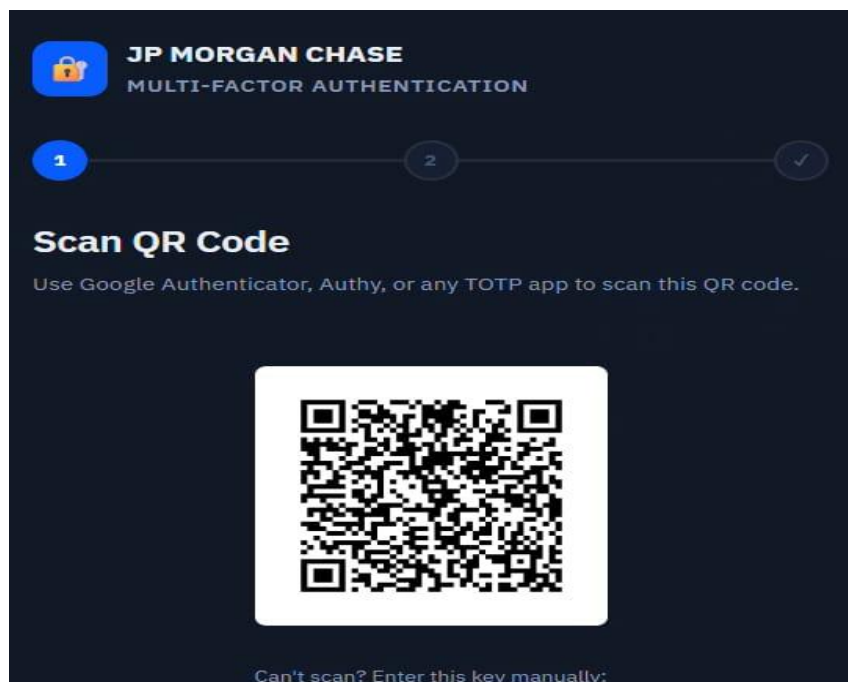
The screenshot shows the 'Sign In' interface for the JP Morgan Chase SIEM Security System. It includes a header with the system name and a sub-header 'Sign In' with a note to 'Enter your credentials to access the secure portal'. The form contains fields for 'USERNAME' (pre-filled with 'morgansystem_01') and 'PASSWORD'. A 'Sign In' button is at the bottom. Below the button, there is a link 'Don't have an account? Create account.' and a footer note: 'This system is monitored. All access attempts are logged and reviewed by the security operations team.'

REGISTRATION Fig(1.3)

5.3 MFA SETUP FLOW

On first login, users are redirected to the MFA setup page, which features a three-step wizard interface. The QR code is generated and displayed once; the tempMfaSecret is stored in the user record but not committed until the user successfully enters the first OTP from their authenticator app. Testing with Google Authenticator and Authy confirmed that the otpauth:// URL format is correctly recognized by both applications.

The OTP verification uses a window parameter of 1, accepting codes valid within one 30-second interval of the current time. This accommodates minor clock drift between the server and the user's mobile device while maintaining strong security. Once confirmed, the mfaEnabled flag is set and the QR code can never be retrieved again — the secret is stored only in hashed JSON, not exposed through any API endpoint.

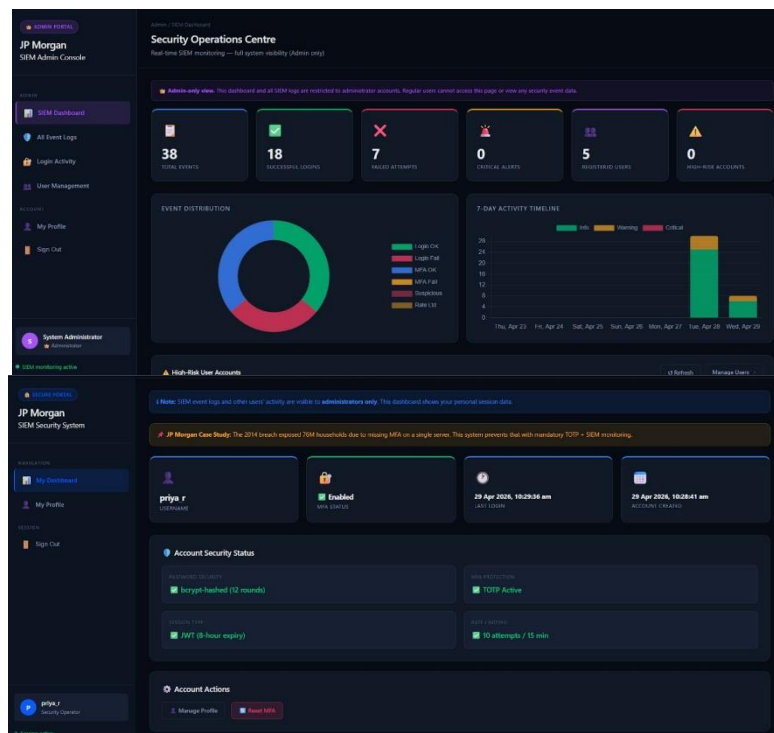


MFA SETUP FLOW fig (1.3)

5.4 SIEM DASHBOARD

The SIEM dashboard provides four key information panels: four metric cards (total events, successful logins, failed attempts, critical alerts), a doughnut chart showing event distribution by type, a 7-day stacked bar chart, and a table of the 10 most recent security events with severity badges. All panels refresh automatically every 10 seconds.

During testing, the dashboard correctly reflected events in real time. Three consecutive failed OTP attempts followed by a rate limit trigger produced the expected SIEM event sequence: three MFA_FAILURE WARNING events followed by a RATE_LIMITED CRITICAL event, all visible in the dashboard within one refresh cycle.



SIEM DASHBOARD fig(1.4)

5.5 SECURITY CONTROL COMPARISON

The following table compares the JP Morgan Chase breach context against the controls implemented in this system:

Failure Mode (JP Morgan)	Traditional System	This System
No MFA on admin server	Password only	Mandatory TOTP MFA on every account
Weak/reused credentials	Plaintext or weak hash	bcrypt 12-round hashing
No brute-force protection	Unlimited attempts	10 attempts/15 min; CRITICAL alert
Breach undetected for weeks	Manual log review	Real-time SIEM with auto-refresh
QR codes re-displayable	N/A	QR issued once, confirmed before commit
No severity classification	Raw log files	INFO / WARNING / CRITICAL taxonomy
No session controls	None or cookie-only	JWT with stage validation (8hr expiry)
No incident dashboard	Absent	Live Chart.js SIEM dashboard

Table 1.1

CHAPTER 6

CONCLUSION AND FUTURE ENHANCEMENTS

6.1 CONCLUSION

The JP Morgan Chase SIEM and MFA Security System successfully demonstrates how layered authentication controls and real-time security monitoring can prevent and detect the class of attack that compromised 76 million accounts in 2014. The system integrates bcrypt password hashing, TOTP multi-factor authentication, JWT session management with stage validation, rate-limited authentication endpoints, and a structured SIEM event logging subsystem into a cohesive, production-quality web application.

Every specific failure mode identified in the JP Morgan breach has a corresponding control in this system. The absence of MFA on a single server — the root cause of the breach — is addressed by mandatory TOTP MFA that cannot be bypassed without the user's registered authenticator device. The weeks-long undetected presence of the attackers is addressed by a SIEM system that logs every authentication event in real time and displays critical alerts on an auto-refreshing dashboard.

The structured event taxonomy with INFO, WARNING, and CRITICAL severity levels provides a foundation for automated alerting and incident response that mirrors the real-world SIEM implementations used by financial institutions post-2014. The rate limiting control, which triggers a CRITICAL SIEM event on threshold breach, ensures that automated brute-force attempts are both blocked and immediately visible to security operators.

6.2 FUTURE ENHANCEMENTS

1. **MongoDB Integration:** Replace the JSON file store with MongoDB to support concurrent multi-user access, indexing, and query optimisation for large-scale SIEM log volumes.
2. **Email OTP as Fallback:** Add SMTP-based OTP delivery as a fallback for users who have lost their authenticator app, reducing support burden while maintaining security.
3. **Push Notification Authentication:** Integrate with services such as Duo Security or Microsoft Authenticator to support push-based MFA approval, improving UX for mobile users.
4. **Machine Learning Anomaly Detection:** Train an anomaly detection model on the SIEM event stream to identify behavioural patterns — unusual login times, geographic anomalies, velocity attacks — beyond what rule-based thresholds can detect.
5. **WebSocket-Based Real-Time SIEM:** Replace the polling-based dashboard refresh with WebSocket connections, providing sub-second event propagation to all connected security operators simultaneously.
6. **Biometric Second Factor:** Integrate WebAuthn/FIDO2 to support hardware security keys (YubiKey) and platform biometrics (fingerprint, face ID) as higher-assurance second factors.
7. **Compliance Reporting:** Add automated generation of PCI-DSS and SOX compliance reports from the SIEM event log, aligned with the regulatory obligations that apply to financial institutions such as JP Morgan Chase.
8. **Container Deployment:** Package the application as a Docker container with environment variable injection, enabling deployment to cloud platforms (AWS ECS, Azure Container Instances) with minimal configuration.

REFERENCES

1. United States v. Gery Sharon et al. (2015). United States District Court, Southern District of New York. Indictment unsealed November 2015.
Link: <https://www.justice.gov/usao-sdny/pr/manhattan-us-attorney-announces-charges-against-hackers-largest-data-breach-history>
2. National Institute of Standards and Technology. (2017). Digital Identity Guidelines — Authentication and Lifecycle Management (NIST SP 800-63B). U.S. Department of Commerce.
Link: <https://pages.nist.gov/800-63-3/sp800-63b.html>
3. RFC 6238 — David M'Raihi et al. (2011). TOTP: Time-Based One-Time Password Algorithm. IETF.
Link: <https://datatracker.ietf.org/doc/html/rfc6238>
4. Niels Provos, & David Mazières. (1999). A Future-Adaptable Password Scheme. USENIX Annual Technical Conference.
Link: <https://www.usenix.org/legacy/events/usenix99/provos/provos.pdf>
5. Dinei Florêncio, & Cormac Herley. (2007). A Large-Scale Study of Web Password Habits. WWW '07. ACM.
Link: <https://dl.acm.org/doi/10.1145/1242572.1242661>
6. Joseph Bonneau et al. (2012). The Quest to Replace Passwords: A Framework for Comparative Evaluation of Web Authentication Schemes. IEEE Symposium on Security and Privacy.
Link: <https://ieeexplore.ieee.org/document/6234423>

7. OWASP Foundation. (2023). Authentication Cheat Sheet.
Link: https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html
8. OWASP Foundation. (2023). Logging Cheat Sheet.
Link: https://cheatsheetseries.owasp.org/cheatsheets/Logging_Cheat_Sheet.html
9. Express.js — OpenJS Foundation. (2024). Express.js Documentation (v4.x).
Link: <https://expressjs.com/>
10. speakeasy Contributors. (2024). speakeasy — Two-factor authentication for Node.js.
Link: <https://github.com/speakeasyjs/speakeasy>
11. Chart.js Contributors. (2024). Chart.js Documentation (v4.4).
Link: <https://www.chartjs.org/docs/latest/>
12. Gartner. (2022). Market Guide for Security Information and Event Management.
Gartner, Inc.
(Note: Full report may require subscription)
Link: <https://www.gartner.com>
13. Ross Anderson. (2008). Security Engineering: A Guide to Building Dependable Distributed Systems (2nd ed.). Wiley.
Link: <https://www.cl.cam.ac.uk/~rja14/book.html>